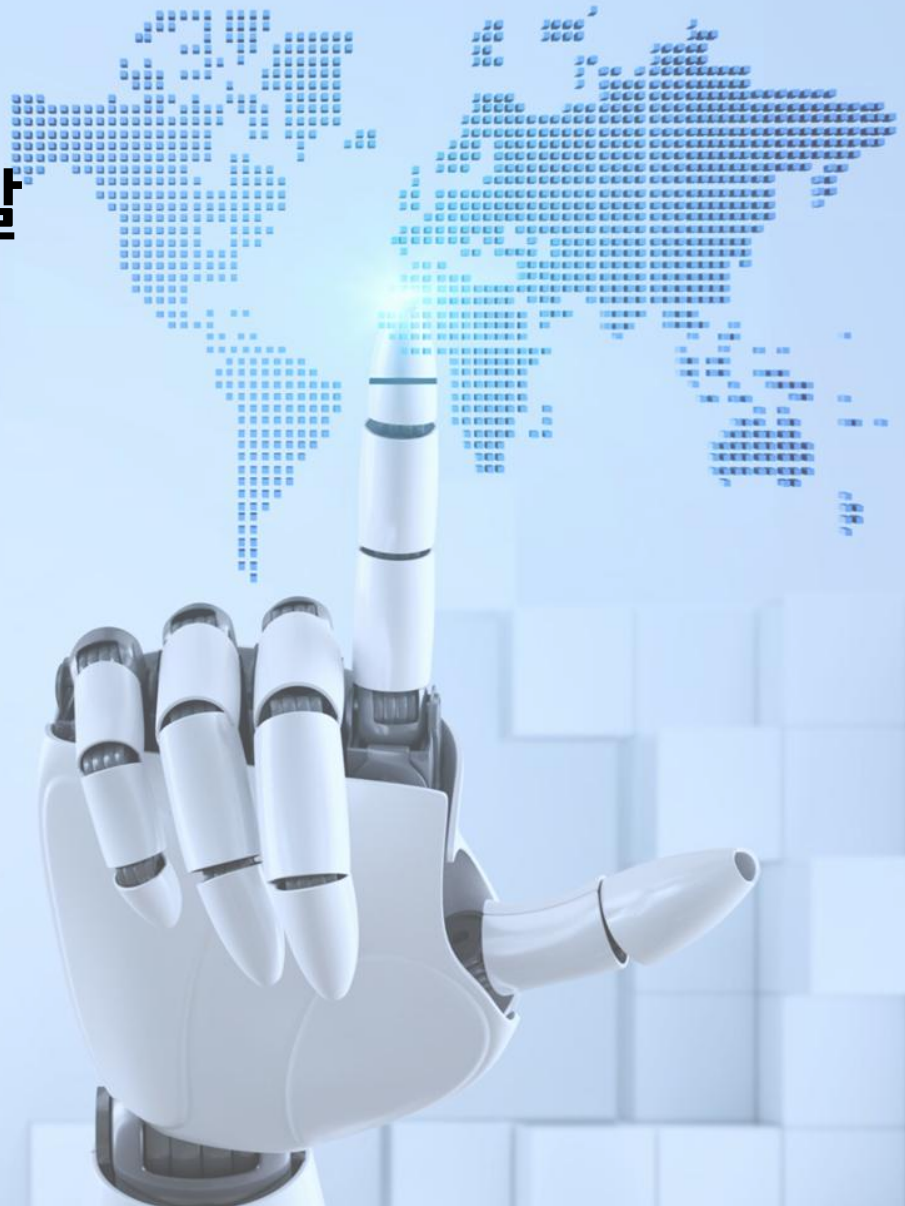
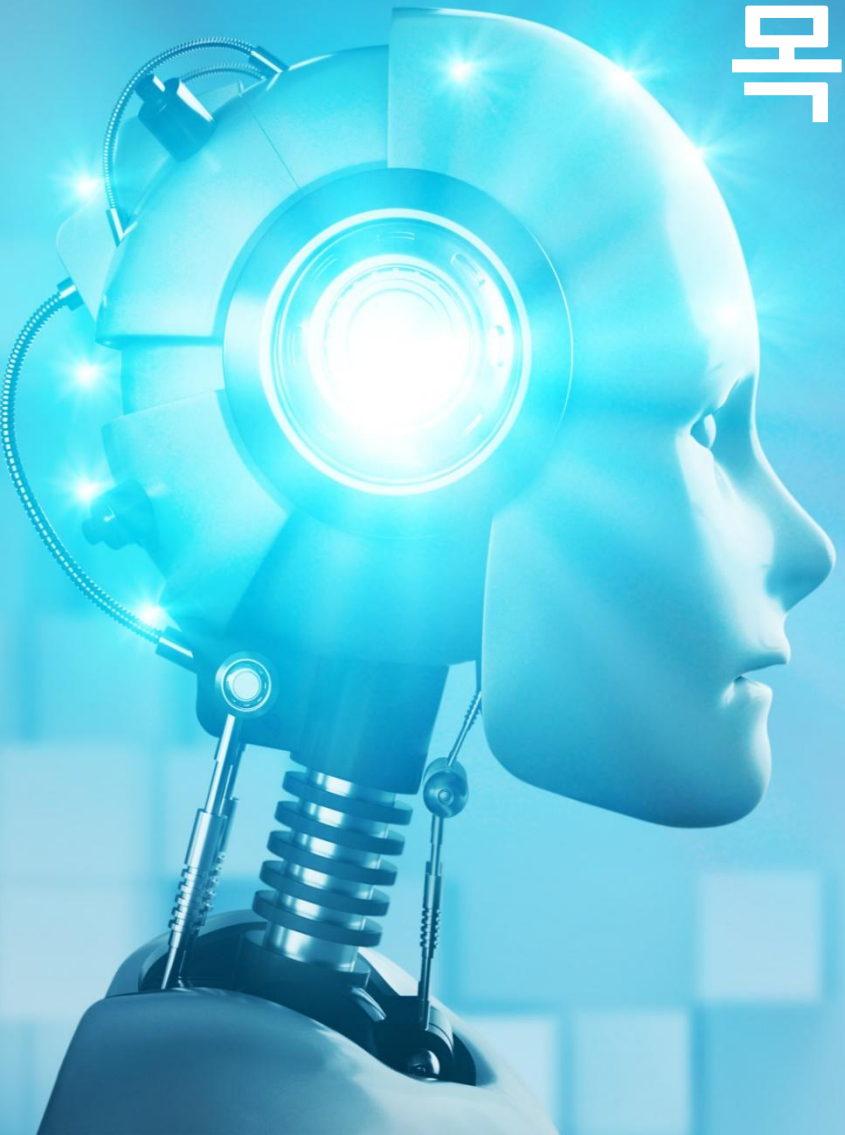
인공지능(AI) 개발자 양성 과정

6 차시

AI 웹애플리케이션 개발

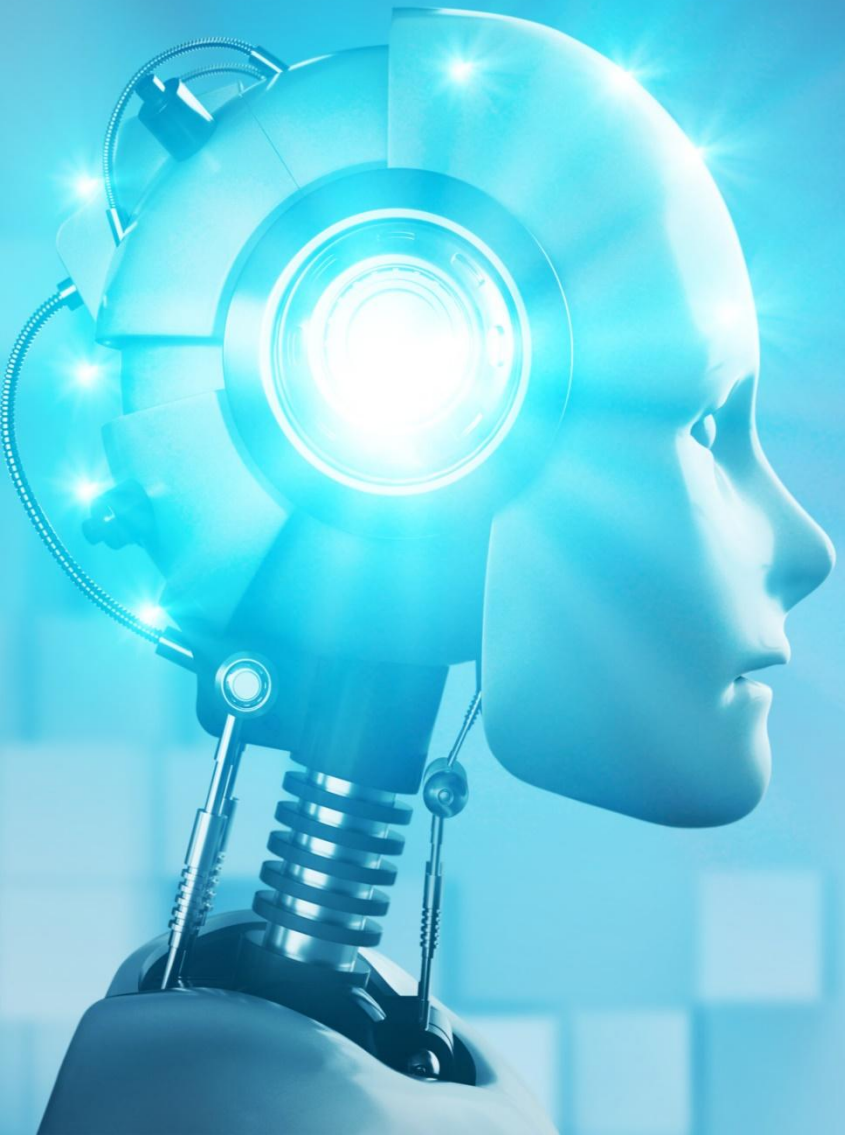


휴먼교육센터 안홍근 강사



목차 Contents

1. Streamlit 기초
2. REST API 연동
3. Gradio 인터페이스 기초



1 Streamlit 기초

1. Steramlit 개요
2. Streamlit 설치방법
3. Streamlit 문법
4. Streamlit 예시

1. Streamlit 개요

1.1 정의와 특징

- Streamlit은 데이터 과학자와 머신러닝 엔지니어를 위한 오픈 소스 파이썬 라이브러리
- 최소한의 코딩으로 대화형 웹 애플리케이션을 빠르게 만들 수 있게 해주는 도구

특징	설명
간편한 개발	Python 스크립트만으로 웹 애플리케이션 개발 가능
빠른 프로토타이핑	복잡한 프론트엔드 지식 없이 빠른 UI 구현
대화형 위젯	슬라이더, 버튼, 텍스트 입력 등 다양한 대화형 요소 제공
데이터 시각화 통합	Matplotlib, Plotly 등 데이터 시각화 라이브러리와 쉬운 통합
머신러닝 모델 시각화	머신러닝 모델 결과를 쉽게 웹 인터페이스로 표현 가능

2. Streamlit 설치방법

설치방법

- pip를 사용한 설치 : pip install streamlit
- conda를 사용한 설치 : conda install –c conda–forge streamlit
- 실행 명령어 : streamlit run 파일이름. py → 실행 후 기본 웹브라우저에서 http://localhost:8501 이 열림

설치 테스트

- streamlit라이브러리 설치 → streamlit_hello.py 파일 생성 → 이메일 등록 → 웹브라우저 'Helllo World' 메시지 확인 (자동으로 열림) → 종료시 (CTRL + C)

streamlit_hello.py

```python
import streamlit as st

st.title('Hello World!')
```

```
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://172.17.0.181:8501
```

3. Streamlit 문법

- Streamlit 앱은 파이썬 함수를 사용하여 구성. 각 함수는 앱의 특정 부분을 나타내며, Streamlit 함수를 호출하여 UI요소 추가
- 코드를 변경하면 자동으로 앱이 업데이트되어 개발 과정을 효율적으로 만들어줌

구분	문법	설명
페이지 구성	st.title()	페이지 제목 설정
	st.header() st.subheader	섹션 제목 설정
	st.tabs()	탭의 이름을 리스트 형태로 받아서 탭으로 표시
데이터표시	st.write()	텍스트, 데이터프레임, 플롯 등 다양한 형태의 데이터 표시
	st.dataframe()	인터팩티브 데이터프레임 표시
	st.table	정적 테이블 표시
대화형 위젯	st.slider()	숫자 선택 슬라이더
	st.button()	클릭 가능한 버튼
	st.selectbox()	드롭다운 선택 메뉴
	st.number_input()	숫자 입력 필드
	st.text_input()	텍스트 입력 필드
시각화	st.pyplot()	Matplotlib 플롯 표시
	st.plotly_chart()	Plotly 차트 표시
	st.bar_chart()	막대 차트
	st.line_chart()	선 그래프
레이아웃	st.sidebard	사이드바 생성
	st.columns()	다중 열 레이아웃
	st.expander()	접고/펼수 있는 섹션

4. Streamlit 예시(Iris 꽃 분류기) (1/4)

- 라이브러리 임포트 : streamlit, pandas, sklearn
- Streamlit 애플리케이션 제목 설정 : st.title()
- Iris 데이터셋 로드 및 전처리 : load_iris() 를 활용하여 Scikit-learn의 내장 Iris 데이터셋 로드

```
# 1. 필요한 라이브러리 임포트
import streamlit as st  # Streamlit 웹 애플리케이션 개발 라이브러리
import pandas as pd  # 데이터 조작 및 분석 라이브러리
from sklearn.datasets import load_iris  # Scikit-learn의 Iris 데이터셋
from sklearn.model_selection import train_test_split  # 데이터 분할 함수
from sklearn.ensemble import RandomForestClassifier  # 랜덤 포레스트 분류기
from sklearn.metrics import accuracy_score  # 정확도 계산 함수

# 2. Streamlit 애플리케이션의 제목 설정
st.title('Iris 꽃 분류기')

# 3. Iris 데이터셋 로드 및 전처리
# load_iris()로 데이터셋 불러오기
iris = load_iris()

# 특징(features) 데이터프레임 생성
# iris.data: 꽃받침 길이, 꽃받침 너비, 꽃잎 길이, 꽃잎 너비
X = pd.DataFrame(iris.data, columns=iris.feature_names)

# 타겟(품종) 시리즈 생성
# iris.target_names[iris.target]: 각 데이터 포인트의 품종 이름으로 변환
y = pd.Series(iris.target_names[iris.target], name='species')
```

4. Streamlit 예시(Iris 꽃 분류기) (2/4)

- 사이드바 하이퍼파라미터 조정 : st.sidebar.slider()를 사용하여 사용자가 실시간으로 모델 하이퍼 파라미터 조정 가능
- 데이터 분할 : train_test_split() 데이터를 학습 및 테스트 세트로 분할, test_size=0.3 전체 데이터의 30% 테스트 세트 사용
- 머신러닝 모델 학습 : RandomForestClassifer() 앙상블 학습 알고리즘, n_estimators 생성할 트리 개수, fit() 모델 학습

```
# 4. 사이드바 생성 및 하이퍼파라미터 조정 위젯
# 사이드바에 머신러닝 모델 하이퍼파라미터 설정 위젯 추가
st.sidebar.header('사용자 입력 파라미터')

# 트리 개수 선택 슬라이더 (1~100, 기본값 10)
n_estimators = st.sidebar.slider('트리 개수', 1, 100, 10)

# 트리 최대 깊이 선택 슬라이더 (1~10, 기본값 3)
max_depth = st.sidebar.slider('최대 깊이', 1, 10, 3)

# 5. 데이터 분할
# train_test_split: 데이터를 학습용(70%)과 테스트용(30%)으로 분할
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 6. 머신러닝 모델 학습
# RandomForestClassifier: 앙상블 학습 방법 중 하나
# n_estimators: 트리 개수, max_depth: 트리 최대 깊이
clf = RandomForestClassifier(n_estimators=n_estimators, max_depth=max_depth, random_state=42)
# 모델 학습
clf.fit(X_train, y_train)
```

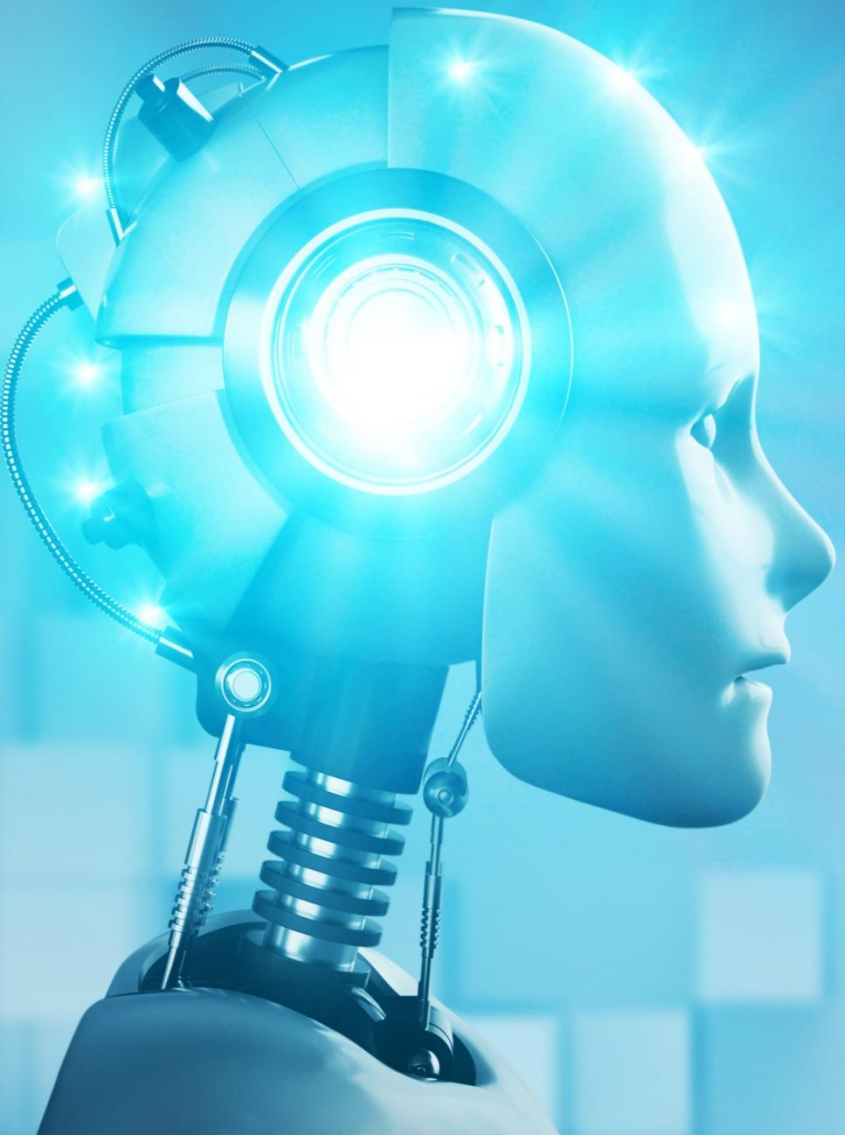
4. Streamlit 예시(Iris 꽃 분류기) (3/4)

- 모델 성능 평가 : predict() 테스트 데이터로 품종 예측, accuracy_score() 예측 정확도 계산
- 결과 시각화 : st.write() 모델 정확도 출력
- 특징 중요도 시각화 : feature_importance 각 특징의 모델 예측 기여도 계산, st.bar_chart() 특징 중요도를 막대그래프로 시각화

```
# 7. 모델 성능 평가
# 테스트 데이터로 예측 수행
y_pred = clf.predict(X_test)

# 정확도 계산
accuracy = accuracy_score(y_test, y_pred)

# 8. 결과 시각화 및 출력
# 모델 정확도 출력
st.write(f'모델 정확도: {accuracy * 100:.2f}%')

# 9. 특징 중요도 시각화
st.subheader('특징 중요도')

# 특징 중요도 DataFrame 생성
feature_importance = pd.DataFrame({
    '특징': X.columns,
    '중요도': clf.feature_importances_
}).sort_values('중요도', ascending=False)

# 막대 차트로 특징 중요도 시각화
st.bar_chart(feature_importance.set_index('특징'))
```

4. Streamlit 예시(Iris 꽃 분류기) (4/4)

- 어떤 꽃 특징(길이, 너비)이 품종 분류에 더 중요한지 직관적으로 이해 가능
- localhost:8501 경로를 통한 시각화



2 REST API 연동

1. FastAPI 개요
2. FastAPI와 스프링부트 연동

1. FastAPI 개요

- 파이썬으로 작성된 Django, Flask에 비해 현대적이고 빠른 웹 프레임워크
- Restful 등 API 개발에 초점을 맞추어 설계되었고, 높은 성능과 직관적인 문법 제공

특징	설명
파이썬 프레임워크	• 파이썬 3.6부터 제공되는 트랜디하고 높은 성능을 가진 파이썬 기반 프레임워크
높은성능	• Pydantic을 활용하여 데이터 검증을 수행하며 매우 빠른 속도가 특징 • ASGI(Asynchronous Server Gateway Interface)를 통한 비동기처리 지원
빠른 코드 작성	• 직관적인 코드 작성이 가능하며, 간결한 문법으로 빠르게 학습 가능
자동 API문서 생성	• OpenAPI(Swagger UI)를 기반으로 자동으로 API 문서 생성 → ex.http:localhost:8000/docs에 접속 시 API 문서 확인 가능 • 개발자가 별도로 문서를 작성할 필요 없음
데이터 검증	• Pydantic을 통한 강력한 데이터 검증 • 입력 데이터의 타입 및 형식 자동 체크
비동기 지원	• 비동기(async/await) 프로그래밍 완전 지원하고 높은 동시성 처리 능력

- Starlette : FastAPI의 기반이 되는 경량 ASGI(비동기 처리 표준 인터페이스)를 기반으로 만들어진 프레임워크
- Pydantic : API 요청/응답 데이터 모델을 정의하고, 이를 통해 잘못된 데이터 형식의 데이터가 들어올 때 오류 발생시키는 라이브러리

2. FastAPI와 스프링부트 연동

2.1 FastAPI 설치방법

설치방법

- Python 3.8 이상 (ex. 본강의 Python 3.10.6 기준)
- pip를 사용한 설치 : pip install fastapi uvicorn pydantic requests # FastAPI, Uvicorn(ASGI 서버), pydantic requests 설치

FastAPI 애플리케이션 실행

- main.py가 위치한 경로로 이동 후에 아래 명령어 실행
- 터미널 창 : unicorn main:app –reload
- 종료 명령어 : CTRL + C

[API 문서확인]

Swagger UI 접속 : http://localhost:8000/docs

```
PS C:\99.참고자료\01.연구개발\zztest\waterPredict_project> uvicorn main:app --reload
INFO:     Will watch for changes in these directories: ['C:\\99.참고자료\\01.연구개발\\zztest
\\waterPredict_project']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [36532] using StatReload
2024-12-11 10:52:27.473888: I tensorflow/core/util/port.cc:153] oneDNN custom operations are
on. You may see slightly different numerical results due to floating-point round-off errors f
rom different computation orders. To turn them off, set the environment variable `TF_ENABLE_O
NEDNN_OPTS=0`.
2024-12-11 10:52:33.688133: I tensorflow/core/util/port.cc:153] oneDNN custom operations are
on. You may see slightly different numerical results due to floating-point round-off errors f
rom different computation orders. To turn them off, set the environment variable `TF_ENABLE_O
NEDNN_OPTS=0`.
INFO:     Started server process [36652]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

2.2 FastAPI와 스프링부트 연동 예시 ▶ 2.2.1 프로젝트 정의 및 특징

- 이 프로젝트는 전력 사용량예측을 위한 FastAPI와 Spring Boot 기반의 웹 애플리케이션으로 REST API 활용
- 파이썬 기반의 머신러닝 모델을 사용하여 향후 7일간의 전력 사용량을 예측하고, 이를 웹 인터페이스를 통해 시각화

목표

- Python 기반의 FASTAPI에서 예측 데이터를 생성 및 제공
- Java 기반의 Spring Boot에서 데이터를 받아 JSP화면으로 전달
- 최종적으로 사용자에게 웹 화면에서 데이터를 시각적으로 보여줌

주요 구성요소

- FASTAPI : 데이터 예측과 REST API 제공을 담당
- Spring Boot : 데이터를 FastAPI에서 가져오고 JSP를 통해 웹 화면으로 전달
- JSP : 데이터를 시각적으로 보여주는 프론트엔드 역할

기술 스택

- 백엔드 : FastAPI(Python), Spring Boot(Java)
- 프론트엔드 : JSP, Chart.js
- 데이터 처리 : sklearn, tensorflow, pandas 등

2.2 FastAPI와 스프링부트 연동 예시 ▶ 2.2.2 REST API 통신 흐름도

데이터 생성 및 예측단계(Python ↔ FastAPI)

- 머신러닝 모델을 통해 전력 사용량 예측
- 예측 데이터를 DataFrame으로 준비
- /predict 엔드포인트를 통해 JSON 데이터 노출

데이터 요청단계(Spring Boot)

- RestTemplate을 사용하여 FastAPI 엔드포인트 호출
- JSON 데이터를 DTO로 변환
- 컨트롤러를 통해 웹 페이지(JSP)로 데이터 전달
- 차트/테이블 렌더링

애플리케이션 실행

- FASTAPI 서버 시작 : http://localhost:8000
- Spring Boot 서버 시작 : http://localhost:8080

```
Python 머신러닝 모델
    ↓ 데이터 예측
FastAPI 서버 (JSON 엔드포인트)
    ↓ REST API
Spring Boot 서버 (데이터 수신 및 변환)
    ↓ 모델 바인딩
JSP 뷰 (데이터 시각화)
    ↓ 렌더링
웹 브라우저
```

2.2 FastAPI와 스프링부트 연동 예시 ▶ 2.2.3 프로젝트 구조

파이썬 FastAPI(waterPredict_project)

- main.py : FastAPI 서버의 진입점
- app/models : 데이터 모델과 데이터 클래스 정의
- app/routers : FastAPI 엔드포인트 라우팅 정의
- app/services : 주요 비즈니스 로직 처리
- app/static : 정적 파일(JS, CSS)을 저장
- app/templates : HTML 템플릿 파일 저장

```
waterPredict_project/        # Python FastAPI 프로젝트 (백엔드 서버)
│
├── main.py                  # FastAPI 실행 파일
├── app/                     # FastAPI 주요 코드
│   ├── __init__.py
│   ├── models/              # 데이터 모델 정의
│   │   ├── __init__.py
│   │   └── prediction.py    # 예측 데이터 모델 정의
│   ├── routers/             # API 라우터
│   │   ├── __init__.py
│   │   └── predict.py       # /predict 엔드포인트 라우팅
│   ├── services/            # 비즈니스 로직
│   │   ├── __init__.py
│   │   └── prediction_service.py  # 예측 처리 로직
│   ├── static/              # 정적 파일 (JS, CSS 등)
│   └── templates/           # HTML 템플릿
│       └── index.html
├── requirements.txt         # Python 패키지 의존성 파일
└── README.md                # 프로젝트 설명
```

2.2 FastAPI와 스프링부트 연동 예시 ▶ 2.2.3 프로젝트 구조

스프링부트

- controller : FastAPI에서 데이터를 받아서 JSP 전달
- dto : FastAPI에서 받은 JSON데이터를 매핑
- jsp : 예측 데이터를 테이블과 차트로 시각화
- application.properties : DB와 서버관련 설정 관리

```
spring_project/              # Spring Boot 프로젝트 (프론트엔드 + API 서버)
│
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/
│   │   │       └── happinessG/
│   │   │           ├── controller/
│   │   │           │   └── PredictionController.java  # FastAPI와 통신하는 컨트롤러
│   │   │           ├── dto/
│   │   │           │   └── PredictionDTO.java         # 예측 데이터 DTO
│   │   │           ├── service/
│   │   │           │   └── PredictionService.java     # 비즈니스 로직
│   │   │           └── repository/
│   │   │               └── PredictionRepository.java  # DB 액세스 레이어
│   │   ├── resources/
│   │   │   ├── static/                                # 정적 파일 (CSS, JS 등)
│   │   │   ├── templates/
│   │   │   │   └── prediction.jsp                     # JSP 템플릿
│   │   │   └── application.properties                 # Spring Boot 설정
│   │   └── webapp/
│   │       └── WEB-INF/
│   │           └── views/
│   │               └── prediction.jsp                 # JSP 파일 위치
```

2.2 FastAPI와 스프링부트 연동 예시 ▶ 2.2.4 주요 동작 과정 (1/5)

REST API 엔드포인트 생성 (main.py)

- @app.get("/predict") : 데코레이터로 REST API 엔드포인트 생성
- predict_water_electricity() : 머신러닝 모델로 예측 수행
- get_forecast_data() : 예측된 데이터 조회
- to_dict(orient='records') : DataFrame을 JSON 배열로 변환 : ex [{"Date": "2024-01-01", "Predicted_wattage":100.5}, ...}

```python
@app.get("/predict")  # REST API GET 엔드포인트 정의
async def predict():
    try:
        # 1. 전력 사용량 예측 수행
        predict_water_electricity()

        # 2. 예측된 데이터 조회
        forecast_data = get_forecast_data()

        # 3. DataFrame을 JSON으로 변환
        return forecast_data.to_dict(orient='records')
    except Exception as e:
        # 오류 발생 시 HTTP 500 에러 반환
        raise HTTPException(status_code=500, detail=str(e))
```

2.2 FastAPI와 스프링부트 연동 예시 ▶ 2.2.4 주요 동작 과정 (2/5)

CORS 미들웨어 설정 (main.py)

- CORS는 다른 도메인(Spring Boot)에서 API 접근 허용
- 보안 설정을 통해 크로스 오리진(cross origin) 요청 관리

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # 모든 오리진 허용
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

2.2 FastAPI와 스프링부트 연동 예시 ▶ 2.2.4 주요 동작 과정 (3/5)

Spring Boot 서비스(PredictionService.java)

- RestTemplate : HTTP 요청을 쉽게 처리하는 Spring 유틸리티
- MappingJackson2HttpMessageConverter: JSON데이터를 Java객체로 변환
- getForObject() : GET요청으로 데이터 조회
- PredictionDTO[] : FastAPI에서 받은 JSON 배열을 DTO 배열로 변환

```java
@Service
public class PredictionService {
    private static final String FASTAPI_URL = "http://localhost:8000/predict";

    public List<PredictionDTO> getPredictions() {
        RestTemplate restTemplate = new RestTemplate();

        // JSON 변환을 위한 메시지 컨버터 추가
        restTemplate.getMessageConverters().add(new MappingJackson2HttpMessageConverter())

        try {
            // FastAPI 엔드포인트에서 데이터 가져오기
            PredictionDTO[] predictions = restTemplate.getForObject(
                FASTAPI_URL,
                PredictionDTO[].class
            );

            // 배열을 리스트로 변환
            return predictions == null ? List.of() : Arrays.asList(predictions);

        } catch (Exception e) {
            // 오류 처리
            System.err.println("Error fetching predictions: " + e.getMessage());
            return List.of();
        }
    }
}
```

2.2 FastAPI와 스프링부트 연동 예시 ▶ 2.2.4 주요 동작 과정 (4/5)

DTO 매핑 (PredictionDTO.java)

- @JsonProperty : JSON키와 Java 필드 연결
- FastAPI의 {"Date": "2024-01-01", "Predicted_wattage": 100.5}를 Java 객체로 자동 변환

```
@Data
public class PredictionDTO {
    @JsonProperty("Date")  // JSON 필드와 Java 필드 매핑
    private String date;

    @JsonProperty("Predicted_wattage")
    private Double predictedWattage;
}
```

2.2 FastAPI와 스프링부트 연동 예시 ▶ 2.2.4 주요 동작 과정 (5/5)

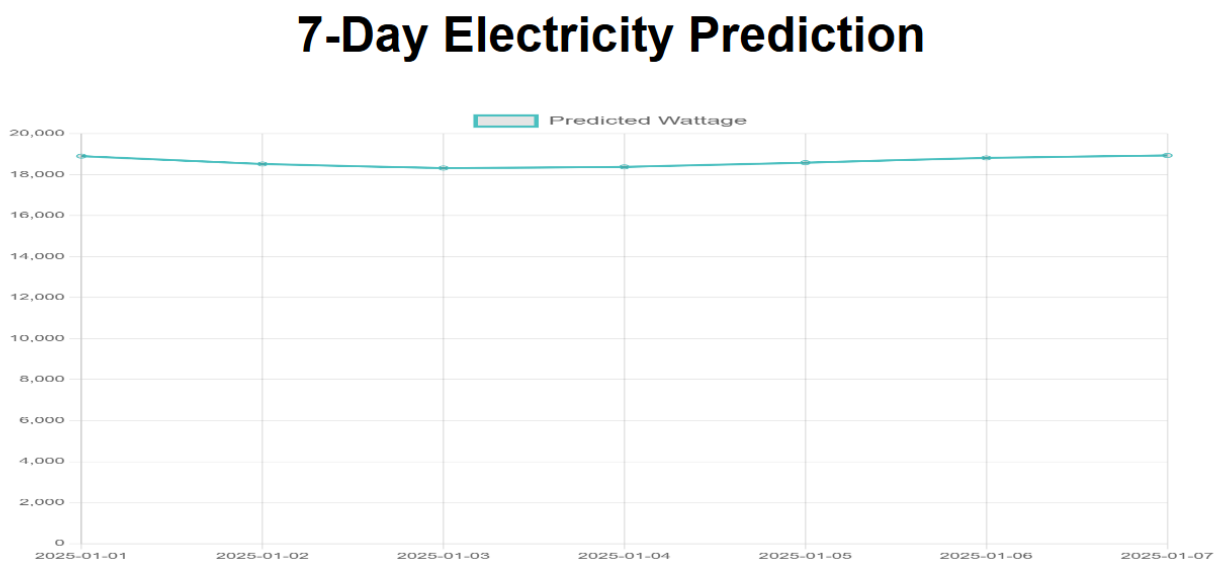
컨트롤러 (PredictionController.java)

- /prediction 요청 시 서비스에서 FastAPI 데이터 조회
- 모델(model)에 데이터를 추가하고, JSP에서 데이터 렌더링

```java
@Controller
public class PredictionController {
    @Autowired
    private PredictionService predictionService;

    @GetMapping("/prediction")
    public String getPredictions(Model model) {
        // 1. 서비스를 통해 예측 데이터 조회
        List<PredictionDTO> predictions = predictionService.getPredictions();

        // 2. 모델에 데이터 추가
        model.addAttribute("predictions", predictions);

        // 3. prediction.jsp로 포워딩
        return "prediction";
    }
}
```

2.2 FastAPI와 스프링부트 연동 예시 ▶ 2.2.5 결과 화면

☑ chart.js와 테이블을 통한 7일간의 전력량 예측 데이터를 보여줌 (http://localhost:8080/prediction)



Date	Predicted Wattage
2025-01-01	18903.64
2025-01-02	18519.75
2025-01-03	18322.73
2025-01-04	18378.9
2025-01-05	18584.33

2.3 참고사항: 비동기 처리 함수 (async/await)

- ☑ async def : 이 함수는 비동기 함수라고 선언하는 키워드
- ☑ await: 비동기 함수 안에서 다른 비동기 작업을 기다리는 키워드
- ☑ asyncio.run(): 이벤트 루프를 돌려서 비동기 코드 실행

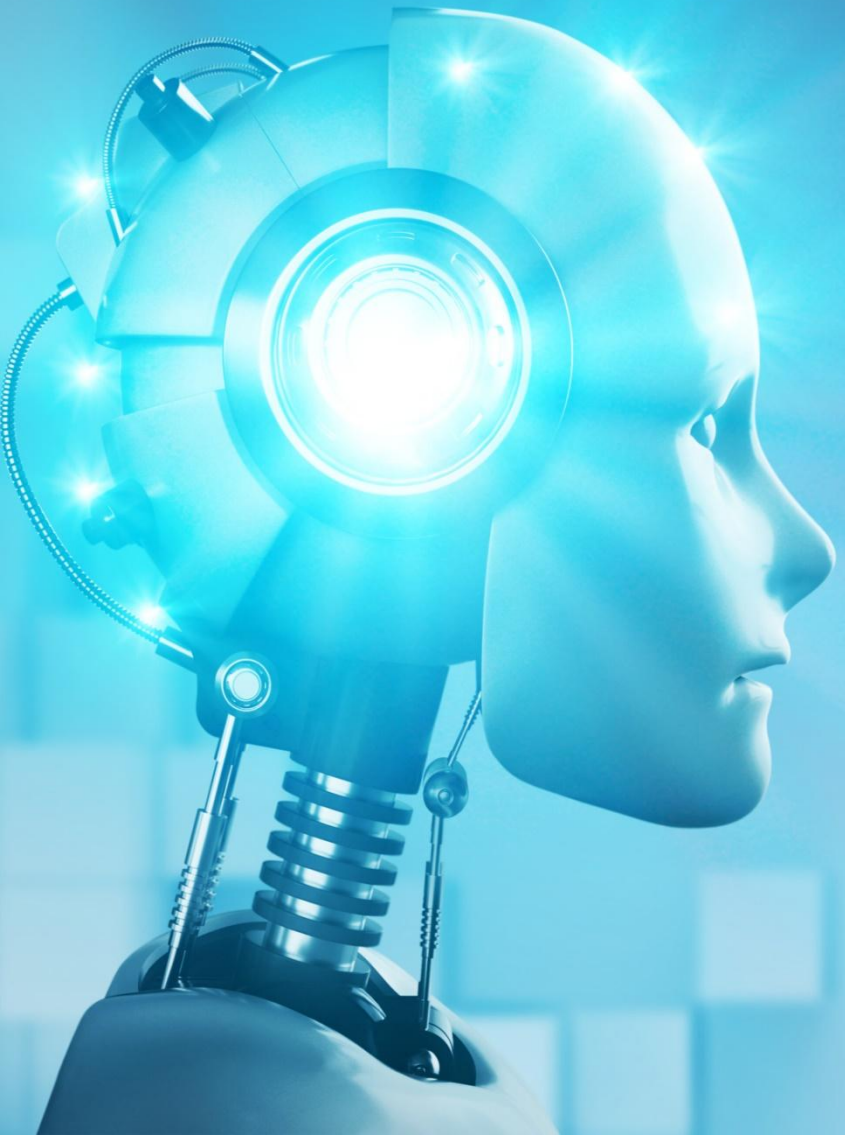
```
import asyncio

async def task(name, delay):
    print(f"{name} 시작")
    await asyncio.sleep(delay)  # delay 동안 CPU 다른 일 가능
    print(f"{name} 완료")

async def main():
    await asyncio.gather(
        task("작업1", 2),
        task("작업2", 1)
    )
asyncio.run(main())
```

**** 작업결과 ****

```
작업1 시작
작업2 시작
작업2 완료
작업1 완료
```

이벤트 루프

- FastAPI가 비동기(async/await)코드를 실행하고, 비동기 작업을 관리하는 작업 스케쥴러
- 작업 대기열을 관리하며 완료된 작업을 순서대로 처리하는 컨트롤 타워 역할
- 이벤트 루프는 작업이 완료될 때까지 기다리지 않고, 다른 작업을 병력적으로 처리해 성능을 높임



3 Gradio 인터페이스 기초

1. Gradio 개요
2. Gradio 설치 및 기본문법

1. Gradio 개요

- 머신러닝 모델이나 파이썬 함수에 대해 **손쉽게 웹 인터페이스를 생성**할 수 있는 **Python 라이브러리**
- 개발/테스트용 인터페이스로 빠르게 시각화하고 공유할 수 있는 강력한 도구로, 초보에서 전문가까지 누구나 쉽게사용 가능

특징	설명
쉬운 인터페이스 생성	• 코드 몇줄로 웹 기반 데모 생성 가능 • 복잡한 웹 개발 지식 불필요 • 머신러닝 모델을 쉽게 공유하고 시연 가능
다양한 입력/출력 지원	• 이미지, 텍스트, 음성, 비디오 등 다양한 데이터 타입 처리 • 다중 입력 및 다중 출력 인터페이스 생성 가능
커스터마이징 용이성	• 테마, 레이아웃, 색상 등 다양한 옵션 제공 • 고급 사용자를 위한 확장성
호스팅 및 공유 기능	• Hugging Face Spaces와 간편한 통합 • 모델 데모를 온라인으로 쉽게 공유 가능

- Hugging Face Spaces : 머신러닝 모델을 누구나 쉽게 배포하고 공유할 수 있는 **오픈 소스 플랫폼**으로, 간단한 설정만으로 자신이 만든 모델을 공개하고, 실시간으로 사용해 볼 수 있도록 만들 수 있음 (url 주소 : https://huggingface.co/spaces)

2. Gradio 설치 및 기본문법

설치방법

- 터미널에서 라이브러리 설치 : pip install gradio

Gradio 기본 문법 및 주요 컴포넌트

구분	내용
인터페이스 생성	• gr.Interface(fn, inputs, outputs)를 사용하여 웹 인터페이스 생성
실행	• interface.launch() 메서드로 웹 앱 실행
입력 컴포넌트	• gr.Textbox() : 텍스트 입력 • gr.Number() : 숫자 입력 • gr.Image() : 이미지 업로드 • gr.Audio() : 오디오 입력 • gr.Video() : 비디오 입력 • gr.Dataframe() : 데이터프레임 입력
출력 컴포넌트	• gr.Textbox() : 텍스트 출력 • gr.Image() : 이미지 출력 • gr.Label() : 분류 결과 출력 • gr. Plot() : 그래프/플롯 출력 • gr.JSON() : JSON 데이터 출력

2. Gradio 설치 및 기본문법

Gradio의 기본 구조

1. 함수 정의

- Gradio에서 수행할 작업(예 : 텍스트 처리, 모델 예측 등)을 정의

2. 인터페이스 정의

- 입력 및 출력 컴포넌트를 설정하고 함수를 연결

3. 실행

- launch() 를 호출하여 Gradio 웹 애플리케이션 실행

4. 종료

- close() 메서드를 호출하여 리소스를 해제하고 앱을 종료

TensorFlow 예제

숫자 두개를 더하는 계산기

```
import gradio as gr

# 두 숫자를 더하는 함수
def add_numbers(a, b):
    return a + b

# 인터페이스 정의
interface = gr.Interface(
    fn=add_numbers,
    inputs=[gr.Number(label="숫자 1"),
            gr.Number(label="숫자 2")],
    outputs=gr.Textbox(label="합계")
)

# 실행
interface.launch()

# 종료
interface.close()
```